

TP C#

JAUGE DE PUISSANCE

AHMED-FATEH GUEZZANE
BOIS-DE-BOULOGNE | ETUDIANT #1174009

Table des matières

Fonctionnalités graphiques	3
Bouton Afficher (1).....	3
Bouton Réinitialiser (2).....	3
Jauge : Composant Graphique (3).....	4
Bouton Sauvegarder (4)	4
Liste des données sauvegardées (5)	5
Bouton Charger (6)	5
Bouton d’affichage du panneau de paramètres (7)	6
Choix de thème (8).....	7
Nombre de divisions (9).....	8
Affichage des sous-divisions (10).....	8
Solution de l’application.....	9
Code du composant graphique (jauge)	10
Attributs	10
Propriétés	10
Méthode OnPaint().....	10
Méthode DrawCalibration	11
Méthode UpdateGauge	11
Code du projet de démarrage	12
Model	12
Model: Date	12
Model: Theme.....	12
Persistence.....	13
Persistence : RegistreData.....	13
Persistence: RegistreThemes	13
DAL : Data.....	14
DAL : DataDAL : IDataDAO (Interface).....	14
DAL : DataDAL : DataDAO_InMemory	14

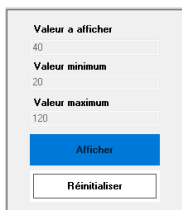
DAL : Theme	15
DAL : ThemeDAL : IThemeDAO	15
DAL : ThemeDAL : ThemeDAO_InMemory	15
Service.....	16
Service : DataService.....	16
Service : ThemeService	16
Form1	17
Conclusion	19

Fonctionnalités graphiques



Bouton Afficher (1)

Ce bouton sert à afficher les données préalablement rentrées par l'utilisateur (valeur à afficher, valeur minimum et valeur maximum) sur le composant graphique.



Une fois que ce bouton est cliqué, le programme valide que les champs ne sont pas vides et remplit la jauge graphique. Il désactive la saisie dans les trois champs (textbox) et désactive également le bouton Afficher puisque les données sont déjà activées.

Bouton Réinitialiser (2)

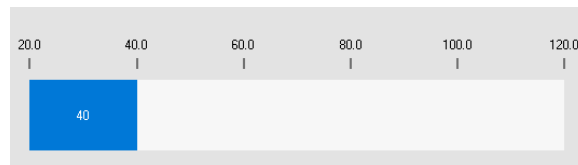
Ce bouton sert à remettre les textbox vides. Une fois que ce bouton est cliqué, le programme met la propriété 'Enable' des TextBoxs et du bouton Afficher à 'true'.

Il appelle également la méthode UpdateGauge du composant graphique en lui envoyant les paramètres par défaut (current value : 0, minimum value : 0, maximum value : 100).

Jauge : Composant Graphique (3)

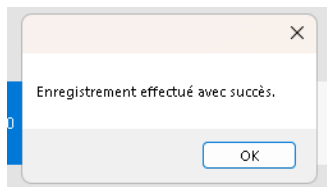
La jauge est un composant graphique qui est inclut dans le projet en tant que référence. Elle contient plusieurs méthodes (sur lesquelles je développerai dans ce document) qui seront appelés dans l'application de la Jauge de puissance.

Principalement, lorsque le bouton "Afficher" ou "Charger" seront cliqués, la jauge se mettra à jour pour représenter les valeurs entrées par l'utilisateur ou celles qui sont chargées.



Bouton Sauvegarder (4)

Le bouton Sauvegarder sauvegarde les données de la jauge (valeur actuelle, valeur minimum, valeur maximum) ainsi qu'un timestamp de la date et de l'heure actuelle. Le bouton appelle une méthode dans la classe service DataService qui ensuite appelle une méthode dans le DAO DataDao_InMemory. Ce DAO accède à un registre en mémoire contenant la liste des données qui ont été sauvegardées.

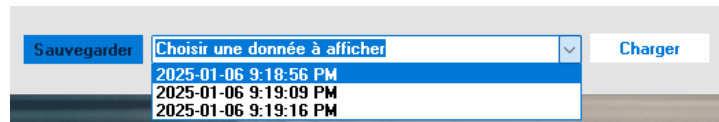


Un message de confirmation est affiché dans un MessageBox confirmant l'ajout de l'enregistrement dans la liste.

Ces classes seront développées dans la section "Code" de ce document.

Liste des données sauvegardées (5)

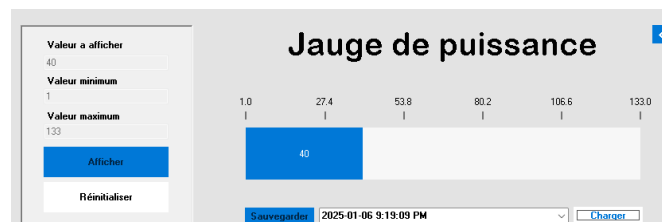
La liste des données sauvegardées est accédée à partir de la classe service DataService qui ensuite appelle une méthode dans le DAO DataDAO_InMemory. Ce DAO accède au registre décrit plus haut contenant la liste des données sauvegardées en mémoire locale.



La liste est chargée en fonction du timestamp des données sauvegardées.

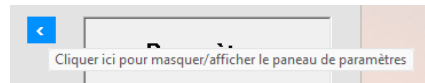
Bouton Charger (6)

Le bouton Charger appelle également une méthode de la classe service DataService qui ensuite appelle à son tour une méthode dans le DAO en mémoire pour le Data. Ce DAO accède au registre en mémoire et trouve l'enregistrement dont le timestamp est égal à celui sélectionné dans la liste. Lorsqu'un enregistrement est chargé, les boutons "Sauvegarder" et "Afficher" se désactivent (afin de ne pas sauvegarder la même donnée deux fois) et les textBox se désactivent également. À ce stade, il est seulement possible de réinitialiser la jauge, charger un nouvel enregistrement ou bien de modifier les paramètres de l'interface sur lesquels je développerai dans ce document.

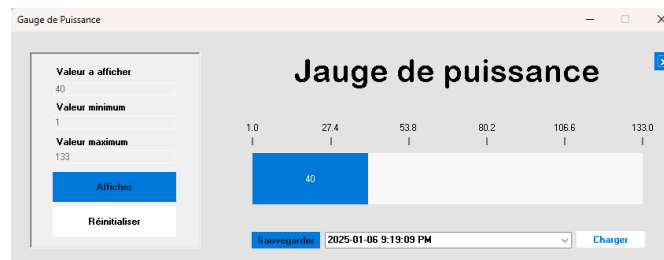


Bouton d'affichage du panneau de paramètres (7)

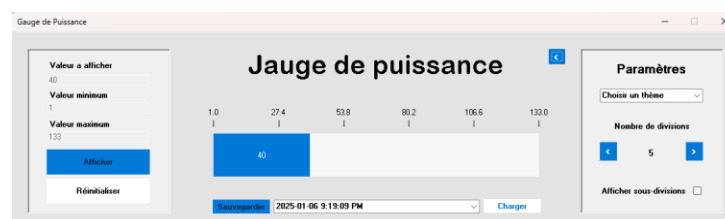
Le bouton d'affichage de paramètre sert à afficher ou bien masquer le panneau de droite de l'interface. Lorsque l'on survole le bouton, un ToolTip nous décrit l'utilité de ce bouton.



En cliquant sur ce bouton, deux comportements peuvent se produire en fonction de l'état actuel d'un flag qui est déclaré dans le Form. Si ce flag est vrai (donc si le panneau de paramètres est visible), le bouton nous servira à fermer ce panneau. Il réduira le Width de l'interface, changera l'icone dans le bouton et désactivera le panneau.



Si le flag est faux (donc si le panneau n'est pas visible), le bouton nous servira à ouvrir ce panneau. Il augmentera le Width de l'interface, changera l'icone du bouton et activera le panneau à nouveau.



NB.: L'interface (Form) a une taille (width et height) fixe. L'utilisateur ne peut donc pas redimensionner l'interface manuellement.

Choix de thème (8)



Le ComboBox de thèmes offre la possibilité à l'utilisateur de personnaliser son interface avec de différentes combinaisons de couleurs. Ce ComboBox est peuplé à partir d'une méthode de la classe service ThemeService qui appelle une méthode du DAO ThemeDAO_InMemory, qui a son tour accède à une liste dans un registre en mémoire.

Voici les thèmes disponibles :

Forest Green



Natural Pink



Light Blue



Gotham Purple



Flaming Red



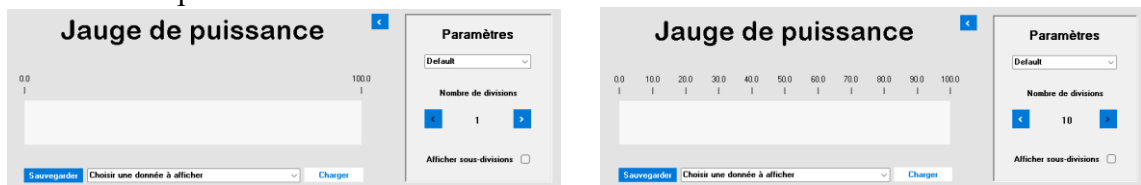
Default



Nombre de divisions (9)

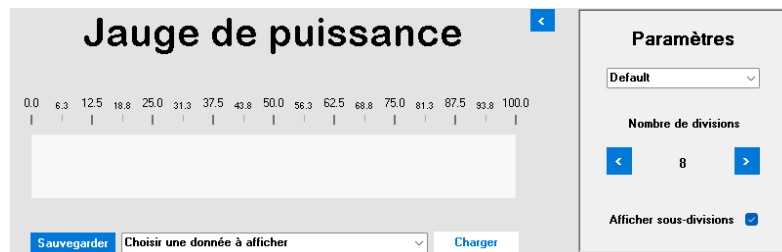
L'option de contrôle du nombre de divisions dans la jauge nous permet de personnaliser notre vue de la jauge.

Le bouton pour diminuer le nombre de division devient désactivé à 1 afin de ne pas avoir de jauge sans aucune division ni données de calibration et il devient également désactivé à 10 afin de ne pas rendre illisible les données de calibration.

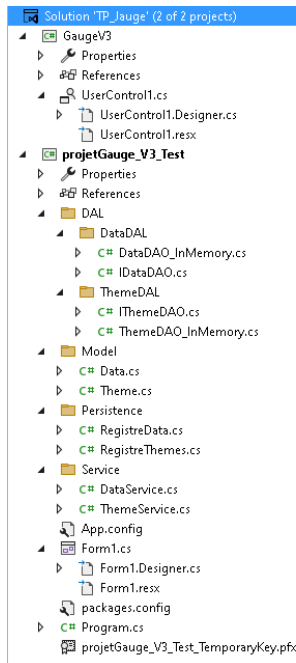


Affichage des sous-divisions (10)

Le checkbox pour afficher les sous-divisions permet à l'utilisateur de personnaliser sa vue de la jauge. A noter qu'activer le checkBox ne rajoute qu'une seule sous-division afin de contrôler la visibilité des valeurs de calibration.



Solution de l'application



Voici un aperçu de la solution TP_Jauge. Elle contient le projet projetGaugeV3_Test mais également un composant graphique (GaugeV3) qui est appelé en référence dans le projetGaugeV3_Test (que l'on appellera dorénavant projet de démarrage).

Le projet de démarrage contient plusieurs dossiers :

- 1) DAL : Le dossier de DAL (Data Access Layer) contient un DAO pour les données ainsi qu'un DAO pour les thèmes.
- 2) Model : Le dossier Model contient les classes Theme et Data afin de créer des objets de ces types pour les opérations CRUD.
- 3) Persistence : Le dossier Persistence contient les registres en mémoire pour les thèmes ainsi que pour les données.
- 4) Service : Le dossier Service contient les classes de service des calculs métiers. Dans notre cas, ces classes de services nous servent à faire le lien entre la vue de l'utilisateur ainsi que les DAO qui accèdent aux données.

Code du composant graphique (jauge)

Attributs

```
// Attributs
#region Attributs
private float _minValue;
private float _maxValue;
private float _currentValue;
private int _numberOfDivisions;
private Color _textColor;
private Color _gaugeHolderColor;
private Color _gaugeColor;
private Color _innerGaugeTextColor;
private bool _showSubDivision;
#endregion
```

Le composant de la jauge comporte les attributs suivants incluant les valeurs de la jauge, les paramètres de calibration ainsi que certains attributs de couleurs pour la jauge.

Propriétés

Chaque attribut représenté par sa propriété qui contient les Get et les Set. Certaines validations sont faites au sein des propriétés comme par exemple si CurrentValue est plus petit que MinimumValue ou plus grande que MaximumValue.

```
#region Properties
[Category("#Values")]
[DefaultValue(0)]
[Description("Valeur minimum sur la gauge")]
12 references
public float MinValue {...}

[Category("#Values")]
[DefaultValue(100)]
[Description("Valeur maximum sur la gauge")]
8 references
public float MaxValue {...}

[Category("#Values")]
[DefaultValue(0)]
[Description("Valeur actuelle a afficher sur la gauge")]
17 references
public float CurrentValue {...}

[Category("#Ruler")]
[DefaultValue(10)]
[Description("Nombre de divisions sur l'echelle de la gauge (Maximum 10)")]
3 references
public int NoOfDivision {...}
```

```
[Category("#Ruler")]
[DefaultValue(false)]
[Description("Set to True if you want to show subdivision")]
1 reference
public bool ShowSubDivision {...}

[Category("#Gauge Colors")]
[Description("Set the color for the Text")]
7 references
public Color TextColor {...}

[Category("#Gauge Colors")]
[DefaultValue("#FF7F7F")]
[Description("Set the color of the gauge holder (background)")]
4 references
public Color GaugeHolderColor {...}

[Category("#Gauge Colors")]
[DefaultValue("Menu Highlight")]
[Description("Set the color of the gauge")]
4 references
public Color GaugeColor {...}

[Category("#Gauge Colors")]
[DefaultValue("White")]
[Description("Set the color of the inner text of the gauge (current value)")]
1 reference
public Color InnerGaugeTextColor {...}
```

Méthode OnPaint()

- 1) La méthode OnPaint fait appel à la méthode DrawCalibration()
- 2) La méthode OnPaint trace ensuite le rectangle représentant la valeur actuelle à afficher.

Méthode OnPaintBackground()

- 1) La méthode OnPaintBackground trace tout d'abord le rectangle GaugeHolder, qui représente le gabarit de la jauge.

Méthode DrawCalibration

Type de retour : void

Name	Description
Graphics g	GDI+ Drawing surface
Rectangle	Rectangle qui comprend la règle de calibration
Int noOfDivision	Nombre de division dans la règle
Bool showSubDivision	Afficher ou non les sous-divisions
Float minValue	Valeur minimum de la règle
Float maxValue	Valeur maximum de la règle

Traitement : Afficher la règle de calibration de la jauge

Méthode UpdateGauge

Type de retour : void

Name	Description
String inputCurrentValue	Valeur a afficher sur la jauge
String inputMinValue	Valeur minimum de la jauge
String inputMaxValue	Valeur maximum de la jauge

Traitement : Accède au Get de chaque valeur ci-dessus et la modifie en fonction des données passées en paramètres.

Code du projet de démarrage

Model

Model: Date

Type	Name
String	_date
Double	_currentValue
Double	_minimumValue
Double	_maximumValue

- Chaque attribut a également sa propriété qui sert de Getter et de Setter.
- Le constructeur assigne la valeur de la Date du system `DateTime.Now.ToString()` à `Data.Date` et prend comme paramètre (double `currentValue`, double `maxVal`, double `minVal`)

Model: Theme

Type	Name
String	_themeName
Color	_gaugeHolderColor
Color	_gaugeColor
Color	_backColor
Color	_buttonColor
Color	_panels

- Chaque attribut a également sa propriété qui sert de Getter et de Setter
- Le Constructeur prend tous les attributs en paramètres

Persistence

Persistence : RegistreData

Type	Name
List<Data>	_dataList
RegistreData	_instance

- L'attribut _dataList a également sa propriété qui sert de Getter et de Setter
- Le design pattern Singleton est appliqué à cette classe afin de ne pas avoir deux instances de RegistreData
- Le constructeur (private) instancie la liste de Data

Persistence: RegistreThemes

Type	Name
List<Theme>	_themes
RegistreThemes	_instance

- L'attribut __themes a également sa propriété qui sert de Getter et de Setter
- Le design pattern Singleton est appliqué à cette classe afin de ne pas avoir deux instances de RegistreThemes
- Le constructeur (private) instancie la liste de Theme et la peuple

DAL : Data

DAL : DataDAL : IDataDAO (Interface)

```
6 references
internal interface IDataDAO
{
    2 references
    void CreateData(Data data);
    2 references
    List<Data> GetDataList();
    2 references
    Data SelectData(string timestamp);
}
```

DAL : DataDAL : DataDAO_InMemory

Nom méthode	Type de retour	Paramètre
CreateData	Void	Data data
GetDataList	List<Data>	N/A
SelectData	Data	String timestamp

- Déclaration de RegistreData _regData
- RegistreData.GetInstance() dans le constructeur

DAL : Theme

DAL : ThemeDAL : IThemeDAO

```
3 references
internal interface IThemeDAO
{
    // signature de la methode GetThemeByKey
    2 references
    Theme GetThemeByTitle(string themeTitle);

    // signature de la methode GetThemeList
    2 references
    List<Theme> GetThemeList();
}
```

DAL : ThemeDAL : ThemeDAO_InMemory

Nom méthode	Type de retour	Paramètre
GetThemeByTitle	Theme	String themeTitle
GetThemeList	List<Theme>	N/A

- Déclaration de RegistreThemes _regTheme
- RegistreTheme.GetInstance() dans le constructeur

Service

Service : DataService

Nom	Type de retour	Paramètre	Appel
GetData	Data	String timestamp	DAO.SelectData(timestamp)
GetDataList	List<Data>	N\A	DAO.GetDataList()
SaveData	Void	Data data	DAO.CreateData(Data)

- Déclaration de l'attribut IDataDAO
- Injection du DAO choisi dans le constructeur (InMemory ou DB)

Service : ThemeService

Nom	Type de retour	Paramètre	Appel
GetThemeByTitle	Theme	String themeTitle	DAO.GetThemeByTitle(string themeTitle)
GetThemes	List<Theme>	N\A	DAO.GetThemeList()

- Déclaration de l'attribut IThemeDAO
- Injection du DAO choisi dans le constructeur (InMemory ou DB)

Form1

Attributs :

```
#region Attributes

//Services
private ThemeService themeService = new ThemeService(new ThemeDAO_InMemory());
private DataService dataService = new DataService(new DataDAO_InMemory());

private bool _settingPanelFlag; //Flag pour le panneau
private static int _divCounter; // Counter pour les divisions
#endregion
```

Nous instancions tout d'abord nos deux services que nous utilisons, celui des thèmes et celui des données. Nous leur passons en paramètres les DAO que nous voulons utiliser (Dependency Injection). Dans notre cas, ce sont des DAO pour des données en mémoire.

Nous avons également un flag pour le panneau de paramètres et un compteur pour le nombre de divisions de la jauge.

Propriétés :

```
#region Properties
3 references
public static int DivCounter
{
    get { return _divCounter; }
    set { _divCounter = value; }
}

5 references
public bool SettingPanelFlag
{
    get { return _settingPanelFlag; } set { _settingPanelFlag = value; }
}
#endregion
```

Le compteur et le flag ont leurs propriétés. Puisque ces deux attributs ne sont utilisés qu'au sein de la classe, il n'était pas vraiment nécessaire de leur donner des getters et setters mais les besoins du programme peuvent changer dans le futur.

Méthodes :

Nom	Retour	Paramètres	Fonction
ChangeDivision	Void	Int	Changer le nombre de divisions de la jauge
SetTheme	Void	String	Changer le theme
PopulateThemeComboBox	Void	N/A	Populer le comboBox des themes
PopulateDataComboBox	void	N/A	Populer le comboBox des données (historique)

Conclusion

Ce travail de fin de session sur la création d'une jauge en C# m'a permis de plonger en profondeur dans le monde des composants graphiques et de renforcer mes connaissances en programmation C#. J'ai acquis une meilleure compréhension des processus impliqués dans la création d'éléments visuels interactifs et des différentes fonctionnalités que C# peut offrir dans ce domaine. La manipulation des objets graphiques, l'utilisation des contrôles standards et la gestion des événements m'ont permis de consolider mes compétences techniques.

Tout au long de ce projet, j'ai également appris de nouveaux concepts et techniques, notamment l'utilisation des bibliothèques graphiques, la gestion des entrées utilisateur et la personnalisation des éléments visuels. Cela a été une véritable opportunité d'apprentissage et m'a permis de repousser mes limites en termes de développement logiciel.

Cependant, si j'avais eu plus de temps, j'aurais aimé ajouter davantage de blocs try et catch pour gérer plus efficacement les exceptions et rendre le programme plus robuste. De plus, j'aurais souhaité explorer davantage les nombreuses possibilités qu'offre C# pour la création d'interfaces graphiques, notamment en ce qui concerne l'intégration d'animations et d'interactions plus complexes. Ces aspects auraient ajouté une dimension supplémentaire à mon projet et m'auraient permis d'enrichir encore mes connaissances en développement d'interfaces utilisateur.

En somme, ce travail m'a été très enrichissant, tant sur le plan technique que créatif, et m'a permis de mieux appréhender les multiples facettes du développement d'applications graphiques avec C#.